

Rational Matrix Policy Optimization for Transformer Language Models

Anonymous authors
Paper under double-blind review

Abstract

Transformer MLP sublayers are usually optimized as ordinary matrix blocks, even though their input matrix, scalar nonlinearity, and output matrix play different roles. We study Rational Latent Basis (RLB), a Transformer sublayer that makes these roles explicit by inserting grouped rational curves between the two matrices, and MatrixPolicy, an optimizer that uses the resulting matrix roles and cheap curve statistics during pretraining. The paper-facing RLB variant is the global-rational/no-local-atom version: each group uses one trainable P5/Q4 rational curve initialized from SiLU, with no local atom path and no atom parameter group. MatrixPolicy applies a manifest-enabled group-gradient policy, a role/depth matrix update with a short early matrix-normalized branch, and a bounded positive rescaling of each input/output matrix pair, while all non-matrix parameters are updated by AdamW. We separate exact properties of the RLB parameterization from heuristic optimizer choices so that the empirical claims can be audited against matched activation and optimizer controls.

1 Method

This section defines the method used for the E1 and E2 paper-facing runs. The activation part is a global-rational RLB sublayer, and the optimizer part is MatrixPolicy. The claim is not that the rational activation alone is sufficient: the matched RLB controls use the same global-rational/no-local-atom sublayer with standard or published optimizers, while the MatrixPolicy row changes only how the RLB matrix roles are optimized.

1.1 Global-Rational RLB

Consider Transformer layer $l \in \{1, \dots, L\}$ with sublayer input $x_l \in \mathbb{R}^d$. RLB replaces the usual Transformer MLP sublayer by a single projected representation of width $H = Gm$, divided into G groups of width m . The input matrix $A_l \in \mathbb{R}^{H \times d}$ produces

$$z_l = A_l x_l, \quad z_l = \text{concat}_{g=1}^G z_{l,g}, \quad z_{l,g} \in \mathbb{R}^m. \quad (1)$$

Each group is normalized by its root-mean-square radius

$$r_{l,g} = \sqrt{m^{-1} \|z_{l,g}\|_2^2 + \epsilon_{\text{rms}}}, \quad u_{l,g} = z_{l,g} / r_{l,g}, \quad (2)$$

where $\epsilon_{\text{rms}} > 0$ is a numerical floor. A scalar rational curve $R_{l,g}$ is then applied coordinate-wise to $u_{l,g}$ and multiplied back by the same radius:

$$h_{l,g} = r_{l,g} R_{l,g}(u_{l,g}), \quad h_l = \text{concat}_{g=1}^G h_{l,g}, \quad y_l = B_l h_l, \quad (3)$$

with output matrix $B_l \in \mathbb{R}^{d \times H}$. The tensor y_l is inserted at the same block location as the standard MLP-sublayer output.

The main RLB variant uses one global rational curve per layer and group,

$$R_{l,g}(u) = \frac{P_{l,g}(u)}{Q_{l,g}(u)}, \quad P_{l,g}(u) = \sum_{i=0}^5 a_{l,g,i} u^i, \quad (4)$$

where

$$Q_{l,g}(u) = 1 + |b_{l,g,1}| |u| + |b_{l,g,2}| u^2 + |b_{l,g,3}| |u|^3 + |b_{l,g,4}| u^4. \quad (5)$$

The coefficients are initialized by a least-squares SiLU fit on a fixed probe interval and are trained with the model. Equation 5 is pole-safe on real inputs because every nonconstant denominator term is nonnegative, so $Q_{l,g}(u) \geq 1$ for all real u . The denominator is piecewise smooth because of the absolute values, which is the intended tradeoff for avoiding real poles during pretraining.

The phrase global rational means that the learned nonlinear response is only the P5/Q4 curve in equation 4 and equation 5. The paper-facing configuration has no local Cauchy atoms, no atom centers, no atom coefficients, and no atom parameter group. MatrixPolicy rows and non-MatrixPolicy RLB optimizer controls use this same activation source.

The RLB matrix pair has a limited positive rescaling identity. Let $D_l(a) = \text{diag}(a_1 I_m, \dots, a_G I_m)$ for $a \in \mathbb{R}_{>0}^G$, let $R_l = \{R_{l,g}\}_{g=1}^G$, and write $f_l(x; A_l, B_l, R_l)$ for the map in equation 1 through equation 3. If $\epsilon_{\text{rms}} = 0$ and all group radii are nonzero, then

$$f_l(x; A_l, B_l, R_l) = f_l(x; D_l(a) A_l, B_l D_l(a)^{-1}, R_l). \quad (6)$$

This identity is not a claim that the full training objective is invariant: weight decay, optimizer moments, finite-precision arithmetic, and the RMS floor all matter in the implemented system. We use it only as a local explanation for why a bounded post-step rescaling of A_l and B_l can control matrix scale without changing the represented block map in the homogeneous idealization.

1.2 MatrixPolicy

MatrixPolicy applies only to the RLB matrix set $\{A_l, B_l\}_{l=1}^L$. Non-matrix parameters, including embeddings, attention weights, normalization parameters, and the rational numerator and denominator coefficients, are updated by the AdamW child optimizer (Kingma & Ba, 2015; Loshchilov & Hutter, 2019). A separate rational coefficient optimizer exists in the code but is inactive in the main method.

MatrixPolicy uses two kinds of statistics. The first kind is gradient pressure. For any tensor V , define the floored RMS

$$\text{rms}_\epsilon(V) = \sqrt{\text{mean}(V^2) + \epsilon}. \quad (7)$$

All RMS quantities used below inside logarithms, denominators, or probe-gain ratios are implemented with the corresponding small positive floor; we write plain rms only when the exact floor value is not a design parameter. For group g , let $A_{l,g} \in \mathbb{R}^{m \times d}$ be the row block of A_l and let $B_{l,g} \in \mathbb{R}^{d \times m}$ be the column block of B_l . With floor $\epsilon_p > 0$, the implemented relative matrix-gradient pressures are

$$p_{l,g}^{\text{in}} = \frac{\text{rms}_{\epsilon_p}(\nabla A_{l,g})}{\max\{\text{rms}_{\epsilon_p}(A_{l,g}), \epsilon_p\}}, \quad p_{l,g}^{\text{out}} = \frac{\text{rms}_{\epsilon_p}(\nabla B_{l,g})}{\max\{\text{rms}_{\epsilon_p}(B_{l,g}), \epsilon_p\}}. \quad (8)$$

For the global-rational variant, the rational activity statistic is

$$p_{l,g}^R = \left[\frac{1}{2} (\text{mean}((\nabla a_{l,g,\cdot})^2) + \text{mean}((\nabla b_{l,g,\cdot})^2)) + \epsilon_p \right]^{1/2}. \quad (9)$$

Let $q_{l,g}^{\text{in}}$, $q_{l,g}^{\text{out}}$, and $q_{l,g}^R$ be exponential moving averages of equation 8 and equation 9 with decay 0.95, floored before logarithms. MatrixPolicy forms

$$\pi_{l,g} = \log q_{l,g}^{\text{in}} - \log q_{l,g}^{\text{out}}, \quad \alpha_{l,g} = \log q_{l,g}^R - \frac{1}{2} (\log q_{l,g}^{\text{in}} + \log q_{l,g}^{\text{out}}). \quad (10)$$

The pressure $\pi_{l,g}$ measures relative update pressure between the input and output matrix roles. The activity $\alpha_{l,g}$ measures whether rational coefficients are receiving large gradients relative to the adjacent matrix pressures.

The second kind of statistic is curve gain. During the activation forward path, RLB caches statistics from a recent sample of normalized coordinates $u_{l,g}$, refreshed every configured statistic interval from at most 512 samples. MatrixPolicy uses the most recent cache:

$$d_{l,g}^{\text{live}} = \text{rms}(R'_{l,g}(u_{l,g})), \quad o_{l,g}^{\text{live}} = \text{rms}(R_{l,g}(u_{l,g})). \quad (11)$$

These quantities are deliberately not full block Jacobians. Derivative gain is a cheap proxy for the input-domain matrix A_l , and output gain is a cheap proxy for the recombination matrix B_l .

Let t be the optimizer step, T the planned number of steps, and $\xi_t = t/T$. The paper-facing manifests enable the group-gradient policy with strengths that are zero by parser default and nonzero in the submitted rows. Its phase is

$$\phi_t = \text{smoothstep}(0.02, 0.30, \xi_t), \quad (12)$$

where $\text{smoothstep}(a, b, \xi)$ is the cubic smoothstep gate from 0 to 1 over $[a, b]$. For positive group values v_j , let $\text{geomean}_j v_j = \exp(G^{-1} \sum_{j=1}^G \log v_j)$. For matrix role $r \in \{\text{in}, \text{out}\}$, the group multiplier is

$$c_{l,g,r} = c_{l,g,r}^{\text{gain}} c_{l,g,r}^{\text{pressure}} c_{l,g}^{\text{activity}}. \quad (13)$$

For the input role, gain uses $d_{l,g}^{\text{live}}$; for the output role, it uses $o_{l,g}^{\text{live}}$:

$$c_{l,g,\text{in}}^{\text{gain}} = \left(\frac{\text{geomean}_j d_{l,j}^{\text{live}}}{d_{l,g}^{\text{live}}} \right)^{0.20\phi_t}, \quad c_{l,g,\text{out}}^{\text{gain}} = \left(\frac{\text{geomean}_j o_{l,j}^{\text{live}}}{o_{l,g}^{\text{live}}} \right)^{0.20\phi_t}. \quad (14)$$

With clipped pressure $\tilde{\pi}_{l,g} = \text{clip}(\pi_{l,g}, -1.50, 1.50)$,

$$c_{l,g,\text{in}}^{\text{pressure}} = \exp(-0.10\phi_t \tilde{\pi}_{l,g}), \quad c_{l,g,\text{out}}^{\text{pressure}} = \exp(+0.10\phi_t \tilde{\pi}_{l,g}). \quad (15)$$

The rational-activity term damps both matrix roles when rational coefficients are unusually active:

$$c_{l,g}^{\text{activity}} = \exp \left[-0.20\phi_t \text{ReLU} \left(\frac{\alpha_{l,g} - 0.05}{0.45} \right) \right]. \quad (16)$$

The multiplier is then recentered and clipped:

$$c_{l,g,r} \leftarrow \frac{c_{l,g,r}}{\text{geomean}_j c_{l,j,r}}, \quad c_{l,g,r} \leftarrow \text{clip}(c_{l,g,r}, 0.75, 1.35). \quad (17)$$

Centering makes the mean log multiplier zero before clipping, so the group rule redistributes matrix update budget instead of silently changing the global learning rate.

After group-gradient scaling, each RLB matrix is updated by a role/depth AdamW branch and a short early matrix-normalized branch. Define normalized depth $\delta_l = (l-1)/(L-1)$ when $L > 1$, with $\delta_l = 1/2$ otherwise. The role factors are

$$\rho_{\text{in}}(l) = \text{clip}(1 - 0.50(\delta_l - 0.5), 0.55, 1.40), \quad \rho_{\text{out}}(l) = \text{clip}(1 + 1.00(\delta_l - 0.5), 0.55, 1.40). \quad (18)$$

The AdamW learning-rate multiplier for role r is

$$s_{l,r}^{\text{Adam}} = \text{clip}(3.0 \max\{0.10, 1 + 1.20(\rho_r(l) - 1)\}, 0.40, 4.0). \quad (19)$$

The early matrix-normalized fraction is reduced when pressure imbalance or rational activity is large. Define

$$\Psi_{l,t} = \frac{1}{G} \sum_{g=1}^G |\text{clip}(\pi_{l,g}, -1.50, 1.50)|, \quad \Omega_{l,t} = \frac{1}{G} \sum_{g=1}^G \text{ReLU} \left(\frac{\alpha_{l,g} - 0.05}{0.45} \right), \quad (20)$$

and

$$\psi_{l,t} = \text{clip}(\exp[-0.30\Psi_{l,t} - 0.65\Omega_{l,t}], 0.10, 1.15). \quad (21)$$

The branch fraction is

$$\mu_{l,r,t} = \text{clip}(0.75 \text{smoothstep}(0.02, 0.12, \xi_t)[1 - \text{smoothstep}(0.20, 0.36, \xi_t)]\rho_r(l)\psi_{l,t}, 0, 0.75). \quad (22)$$

For matrix $M_{l,r}$, where $M_{l,\text{in}} = A_l$ and $M_{l,\text{out}} = B_l$, the implementation composes two stateful child updates:

$$\begin{aligned} \widetilde{M}_{l,r}^{t+1} &= \text{AdamW}_{\eta_t s_{l,r}^{\text{Adam}}(1-\mu_{l,r,t})}(M_{l,r}^t; \nabla M_{l,r}^t), \\ M_{l,r}^{t+1} &= \text{MatNorm}_{\eta_t \mu_{l,r,t}}(\widetilde{M}_{l,r}^{t+1}; \nabla M_{l,r}^t). \end{aligned} \quad (23)$$

Here η_t is the base learning rate, and MatNorm denotes the matrix-normalized branch with momentum and Newton-Schulz normalization. The formula describes operational order, not an additive closed-form optimizer. After $\mu_{l,r,t}$ is permanently zero for all matrix groups, the implementation skips the zero-learning-rate matrix-normalized child step; this is a runtime-only optimization.

Finally, the wrapper applies a bounded positive rescaling to each group of the matrix pair. On a fixed probe grid $u \in [-5, 5]$, define

$$d_{l,g}^{\text{probe}} = \text{rms}(R'_{l,g}(u)), \quad o_{l,g}^{\text{probe}} = \text{rms}(R_{l,g}(u)). \quad (24)$$

Let the current log matrix ratio be

$$\gamma_{l,g} = \log \text{rms}(A_{l,g}) - \log \text{rms}(B_{l,g}). \quad (25)$$

The target log ratio is the heuristic

$$\tau_{l,g} = \frac{1}{2} \left(\log d_{l,g}^{\text{probe}} - \log o_{l,g}^{\text{probe}} \right) + 0.25 \bar{\pi}_{l,g}, \quad \bar{\pi}_{l,g} = \text{clip}(\pi_{l,g}, -1.25, 1.25). \quad (26)$$

Rational activity modulates the strength:

$$a_{l,g}^{\text{rs}} = \exp(\text{clip}(0.10 \bar{\alpha}_{l,g}, \log 0.75, \log 1.25)), \quad \bar{\alpha}_{l,g} = \text{clip}(\alpha_{l,g}, -1.25, 1.25). \quad (27)$$

With

$$s_{l,t} = 0.50 \text{smoothstep}(0.03, 0.35, \xi_t) \text{clip}(1 + 0.15(\delta_l - 0.5), 0.75, 1.25), \quad (28)$$

the log move is

$$\ell_{l,g,t} = \text{clip}\left(\frac{1}{2}[\tau_{l,g} - \gamma_{l,g}]s_{l,t}a_{l,g}^{\text{rs}}, -0.030, 0.030\right), \quad (29)$$

and the matrices are updated by

$$A_{l,g} \leftarrow e^{\ell_{l,g,t}} A_{l,g}, \quad B_{l,g} \leftarrow e^{-\ell_{l,g,t}} B_{l,g}. \quad (30)$$

When compatible optimizer moment tensors exist, their states are scaled covariantly. In the homogeneous idealization of equation 6, equation 30 changes only the representative of the same block map. With the RMS floor, the move is approximate, which is why it is clipped and run only as a bounded post-step correction. The pressure term in equation 26 is a relative-update heuristic, not an exact quotient-space gradient.

Algorithm 1: MatrixPolicy step for global-rational RLB.

1. Update matrix-pressure and rational-activity EMAs from current gradients.
2. Use the most recent cached curve-gain statistics from the RLB forward path.
3. Step non-matrix parameters, including rational coefficients, with the AdamW child optimizer.
4. In the matrix child optimizer, apply the manifest-enabled group-gradient multiplier equation 17 to RLB matrix gradients.
5. Step RLB matrices with the role/depth AdamW branch and the early matrix-normalized branch in equation 23.
6. Apply the bounded positive rescaling in equation 30.

1.3 Reproducibility Boundary

The active paper-facing configuration is manifest-first. The MatrixPolicy rows use `rlb_fused_global_rational` with grouped P5/Q4 rational curves, SiLU initialization, and no local atoms. The backbone and rational coefficients use AdamW. The RLB input and output matrices use the MatrixPolicy child optimizer, with group-gradient policy enabled by the manifest flags `-rational-matrix-policy-group-gain-strength 0.20`, `-rational-matrix-policy-group-pressure-strength 0.10`, `-rational-matrix-policy-group-activity-damping 0.20`, `-rational-matrix-policy-group-start 0.02`, `-rational-matrix-policy-group-end 0.30`, `-rational-matrix-policy-group-min-scale 0.75`, and `-rational-matrix-policy-group-max-scale 1.35`. The parser defaults for the three strengths are zero, and the default group end/clip values differ, so reproducing the paper rows requires the manifest settings.

The active numerical surface is as follows. Matrix AdamW uses the multiplier in equation 19. The matrix-normalized branch uses maximum fraction 0.75, momentum 0.95, five Newton-Schulz steps, and the schedule in equation 22; its final and minimum fractions are both zero. The group-gradient policy uses pressure clip 1.50, activity target 0.05, activity width 0.45, activation window 0.02 to 0.30 of training progress, and multiplier clip [0.75, 1.35]. The bounded positive rescaling runs every five optimizer steps, uses 129 probe points on $[-5, 5]$, target weight 1.0, strength 0.50, activation window 0.03 to 0.35, pressure coefficient 0.25, pressure clip 1.25, activity coefficient 0.10, activity-gain clamp [0.75, 1.25], and maximum absolute log step 0.030.

The following branches are inactive in the main method: the separate rational coefficient optimizer, coefficient Gram preconditioning, transport/quotient projection, transport matrix gain preconditioning, and transport pressure preconditioning. They remain research hooks in the codebase, but they are not part of the reported E1/E2 method unless a manifest explicitly enables them.

References

- Xiangning Chen, Chen Liang, Da Huang, Esteban Real, Kaiyuan Wang, Hieu Pham, Xuanyi Dong, Thang Lu, Cho-Jui Hsieh, Yifeng Lu, and Quoc V. Le. Symbolic Discovery of Optimization Algorithms. In *Advances in Neural Information Processing Systems*, 2023. URL https://papers.neurips.cc/paper_files/paper/2023/hash/9a39b4925e35cf447ccba8757137d84f-Abstract-Conference.html.
- Aaron Defazio, Xingyu Yang, Ahmed Khaled, Konstantin Mishchenko, Harsh Mehta, and Ashok Cutkosky. The Road Less Scheduled. In *Advances in Neural Information Processing Systems*, 2024. URL <https://neurips.cc/virtual/2024/poster/96925>.
- Diederik P. Kingma and Jimmy Ba. Adam: A Method for Stochastic Optimization. In *International Conference on Learning Representations*, 2015. URL <https://mlanthology.org/iclr/2015/kingma2015iclr-adam/>.
- Jeffrey Li, Alex Fang, Georgios Smyrnis, Maor Ivgi, Matt Jordan, Samir Gadre, Hritik Bansal, Etash Guha, et al. DataComp-LM: In Search of the Next Generation of Training Sets for Language Models. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*, 2024. doi: 10.52202/079017-0455. URL https://proceedings.neurips.cc/paper_files/paper/2024/hash/19e4ea30dded58259665db375885e412-Abstract-Datasets_and_Benchmarks_Track.html.
- Kaizhao Liang, Lizhang Chen, Bo Liu, and Qiang Liu. Cautious Optimizers: Improving Training with One Line of Code. In *International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=zBPZeRjfgu>.
- Hong Liu, Zhiyuan Li, David Hall, Percy Liang, and Tengyu Ma. Sophia: A Scalable Stochastic Second-order Optimizer for Language Model Pre-training. In *International Conference on Learning Representations*, 2024. URL https://proceedings.iclr.cc/paper_files/paper/2024/hash/06960915ba8674c7a898ec0b472b80ff-Abstract-Conference.html.

- Ilya Loshchilov and Frank Hutter. Decoupled Weight Decay Regularization. In International Conference on Learning Representations, 2019. URL <https://openreview.net/forum?id=Bkg6RiCqY7>.
- Yang Luo, Xiaozhe Ren, Zangwei Zheng, Zhuo Jiang, Xin Jiang, and Yang You. CAME: Confidence-guided Adaptive Memory Efficient Optimization. In Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics, pp. 4442–4453, 2023. doi: 10.18653/v1/2023.acl-long.243. URL <https://aclanthology.org/2023.acl-long.243/>.
- Matteo Pagliardini, Pierre Ablin, and David Grangier. The AdEMAMix Optimizer: Better, Faster, Older. In International Conference on Learning Representations, 2025. URL <https://iclr.cc/virtual/2025/poster/28625>.
- Guilherme Penedo, Hynek Kydlicek, Loubna Ben Allal, Anton Lozhkov, Margaret Mitchell, Colin Raffel, Leandro von Werra, and Thomas Wolf. The FineWeb Datasets: Decanting the Web for the Finest Text Data at Scale. In Advances in Neural Information Processing Systems, Datasets and Benchmarks Track, 2024. doi: 10.52202/079017-0970. URL https://papers.nips.cc/paper/2024/hash/370df50ccfd8bde18f8f9c2d9151bda-Abstract-Datasets_and_Benchmarks_Track.html.
- Luca Soldaini, Rodney Kinney, Akshita Bhagia, Dustin Schwenk, David Atkinson, Russell Authur, Ben Bogin, Khyathi Chandu, et al. Dolma: An Open Corpus of Three Trillion Tokens for Language Model Pretraining Research. arXiv preprint arXiv:2402.00159, 2024. URL <https://arxiv.org/abs/2402.00159>.
- Nikhil Vyas, Depen Morwani, Rosie Zhao, Itai Shapira, David Brandfonbrener, Lucas Janson, and Sham Kakade. SOAP: Improving and Stabilizing Shampoo using Adam for Language Modeling. In International Conference on Learning Representations, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/e988664070e9591f93fdcf605f7dc623-Abstract-Conference.html.
- Kaiyue Wen, David Leo Wright Hall, Tengyu Ma, and Percy Liang. Fantastic Pretraining Optimizers and Where to Find Them. In International Conference on Learning Representations, 2026. URL <https://openreview.net/forum?id=2J51qUZ0iG>.
- Yushun Zhang, Congliang Chen, Ziniu Li, Tian Ding, Chenwei Wu, Diederik P. Kingma, Yinyu Ye, Zhi-Quan Luo, and Ruoyu Sun. Adam-mini: Use Fewer Learning Rates To Gain More. In International Conference on Learning Representations, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/45ae878717399e6f62d57c65f052cd46-Abstract-Conference.html.
- Jiawei Zhao, Zhenyu Zhang, Beidi Chen, Zhangyang Wang, Anima Anandkumar, and Yuan-dong Tian. GaLore: Memory-Efficient LLM Training by Gradient Low-Rank Projection. In International Conference on Machine Learning, 2024. URL <https://openreview.net/forum?id=hYHsrKDiX7>.

A Exact Mathematical Checks

Pole-safe denominator. For one group, the denominator

$$Q(u) = 1 + |b_1| |u| + |b_2| u^2 + |b_3| |u|^3 + |b_4| u^4 \quad (31)$$

satisfies $Q(u) \geq 1$ for every real u , independently of the raw signs of b_i . Thus $P(u)/Q(u)$ has no real pole. The absolute values make the curve piecewise smooth; this is a numerical design choice, not a theorem about global optimization.

Positive rescaling identity. Set $\epsilon_{\text{rms}} = 0$ and consider a nonzero group. For any $a > 0$,

$$r(az) = \sqrt{m^{-1} \|az\|_2^2} = ar(z), \quad u(az) = \frac{az}{r(az)} = \frac{z}{r(z)} = u(z). \quad (32)$$

Therefore

$$h(az) = r(az)R(u(az)) = ar(z)R(u(z)) = ah(z). \quad (33)$$

Scaling the matching output columns by a^{-1} leaves the group contribution to $B_l h_l$ unchanged. Applying this argument independently to all groups proves equation 6. The identity requires positive scaling; negative scaling would flip the normalized coordinate and is not generally a symmetry of a learned rational curve.

Effect of the RMS floor. With $\epsilon_{\text{rms}} > 0$, scaling a group gives

$$u_\epsilon(az) = \frac{az}{\sqrt{a^2 m^{-1} \|z\|_2^2 + \epsilon_{\text{rms}}}} = \kappa(a, z) u_\epsilon(z), \quad (34)$$

where

$$\kappa(a, z) = \frac{a \sqrt{m^{-1} \|z\|_2^2 + \epsilon_{\text{rms}}}}{\sqrt{a^2 m^{-1} \|z\|_2^2 + \epsilon_{\text{rms}}}}. \quad (35)$$

When $m^{-1} \|z\|_2^2 \gg \epsilon_{\text{rms}}$, $\kappa(a, z) \approx 1$ and the homogeneous identity is a good local model. Near the floor, $\kappa(a, z)$ can differ from 1, so the implemented rescaling is not exactly function-preserving. This is the reason for the small log-step clip in equation 29; the paper does not claim exact invariance for the stabilized implementation.

B Why These Signals Are Plausible but Not Proofs

The output matrix has a direct first-order effect

$$\delta y_{l,g} = \delta B_{l,g} h_{l,g}. \quad (36)$$

Because $h_{l,g} = r_{l,g} R_{l,g}(u_{l,g})$, the RMS of $R_{l,g}(u_{l,g})$ is a cheap proxy for how strongly the group feeds the recombination matrix. This motivates the output gain $o_{l,g}$. The input matrix changes the pre-normalized coordinate. In the homogeneous-radius case, the differential of $h(z) = r(z)R(z/r(z))$ contains

$$dh = dr R(u) + r \text{diag}(R'(u)) du, \quad (37)$$

along with terms from the RMS normalization. MatrixPolicy does not estimate the full Transformer-block Jacobian. It uses the scalar derivative RMS $d_{l,g}$ as a stable, low-cost input-side sensitivity proxy and recenters the resulting inverse-gain multiplier across groups.

The pressure statistic is also heuristic. In the exact homogeneous model, the infinitesimal transform $A_{l,g} \mapsto e^s A_{l,g}$ and $B_{l,g} \mapsto e^{-s} B_{l,g}$ leaves the block map unchanged, so an unregularized loss would have zero derivative along that direction:

$$\langle \nabla_{A_{l,g}} \mathcal{L}, A_{l,g} \rangle_F - \langle \nabla_{B_{l,g}} \mathcal{L}, B_{l,g} \rangle_F = 0. \quad (38)$$

The implementation intentionally does not use this signed inner product. It uses the RMS ratio in equation 8, which is less sensitive to cancellation and closer to the scale of an optimizer update. Therefore $\pi_{l,g}$ should be read as relative update pressure, not as an exact quotient gradient or a natural-gradient estimate.

The early matrix-normalized branch is motivated by training dynamics rather than an invariance theorem. Early in training, the RLB matrices decide which normalized coordinates the rational curves see and how those features return to the residual stream. A temporary matrix-normalized direction update can change these domains quickly. Later, after the rational curves, attention layers, and output recombinations have co-adapted, keeping that branch active adds runtime and can obscure the role/depth AdamW policy. MatrixPolicy therefore schedules the branch early and decays it to zero through equation 22.

Removing local atoms is both an a priori and an a posteriori choice for this paper. A priori, the global-rational variant is the cleaner object for an optimizer study: each group has one rational curve, one input matrix block, and one output matrix block, with no extra atom tensor that could introduce a separate optimizer failure mode. A posteriori, all MatrixPolicy and non-MatrixPolicy RLB optimizer controls in the paper-facing E1/E2 tables use the same global-rational/no-local-atom activation source, so performance differences are attributed to optimizer behavior rather than to a local atom path.

C Runtime and Failure Accounting

Runtime tables should report clean training-process time from completed JSONL summary records. Queue wait, dependency wait, token-cache construction, extension compilation, launcher overhead, and discarded pre-restart partial attempts are not optimizer runtime. If a completed artifact cannot reconstruct trusted per-row training time after preemption, the row must be rerun or marked as unrepaired rather than silently averaged into the runtime table.

Failures are not excluded from the scientific record. If an optimizer/activation pair diverges or early-stops under the stated protocol, it is reported as diverged or early-stopped under the same stopping rule. The accounting rule only separates training behavior from systems artifacts such as preempted allocations or known-bad timing nodes.